

Laboratorium: klient → Nginx → backend

Cel

Uruchomić realistyczny tor ruchu HTTP/HTTPS z własnym CA i reverse proxy.

```
klient
  | HTTP :8080 / HTTPS :8443
  ▼
Nginx
  | reverse proxy /api/
  ▼
backend:9000 (tylko sieć
Dockera)
```

Narzędzia

Docker + Docker Compose
openssl
curl
przeglądarka + DevTools

Ważne porty

8080 = HTTP
8443 = HTTPS
backend:9000 nie ma portu hosta

```
Summary: 17 passed, 4 failed, 0 warnings
• joanna@joanna-VirtualBox:~/lab-nginx-http$ ./scripts/reload_nginx.sh && ./scripts/check_lab.sh
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
2026/08/26 15:33:29 [notice] 86#86: signal process started
[PASS] Command available: docker
[PASS] Command available: openssl
[PASS] Command available: curl
[PASS] Docker daemon is available
[PASS] Certificate files exist
[PASS] Server certificate verifies against local CA
[PASS] Server certificate contains DNS SAN: localhost
[PASS] Server certificate contains DNS SAN: reverse-proxy.lab
[PASS] Server certificate contains IP SAN: 127.0.0.1
[PASS] Nginx container is running
[PASS] Backend container is running
[PASS] Nginx configuration test passes
[PASS] HTTP health endpoint responds
[PASS] Nginx serves the static HTTP page
[PASS] Static asset has expected Cache-Control header
[PASS] Backend is not directly exposed on host port 9000
[PASS] HTTPS endpoint validates with the local CA
[PASS] Reverse proxy reaches the backend service
[PASS] Reverse proxy forwards X-Forwarded-Proto
[PASS] Reverse proxy adds X-Lab-Proxy header
[PASS] Upstream X-Powered-By header is hidden by Nginx

Summary: 21 passed, 0 failed, 0 warnings
• joanna@joanna-VirtualBox:~/lab-nginx-http$
```

Sprawozdanie z wykonania laboratorium: Nginx Reverse Proxy & TLS

1. Cel zadania Skonfigurowanie serwera Nginx jako Reverse Proxy dla aplikacji backendowej, wdrożenie szyfrowania TLS z użyciem własnego lokalnego urzędu certyfikacji (CA) oraz spełnienie wymagań bezpieczeństwa zawartych w instrukcji `README.md`.

2. Przebieg realizacji krok po kroku

- **Pobranie i uruchomienie środowiska:** Rozpakowano repozytorium laboratorium, uruchomiono VS Code (`code .`) oraz wystartowano kontenery Dockerowe za pomocą skryptu pomocniczego `./scripts/start_lab.sh`.
- **Konfiguracja Reverse Proxy i nagłówków (`lab_locations.conf`):**
Zrealizowano wytyczne z komentarzy `# TODO` zawartych w pliku konfiguracyjnym:
 1. Skonfigurowano blok `location /api/` przekierowujący ruch na serwer backendowy (`http://backend:9000/`).
 2. Dodano przekazywanie nagłówków: `Host`, `X-Real-IP`, `X-Forwarded-For` oraz `X-Forwarded-Proto`.
 3. Ukryto nagłówek informacyjny backendu (`proxy_hide_header X-Powered-By`).
 4. Wdrożono nagłówek identyfikacyjny proxy (`add_header X-Lab-Proxy nginx-reverse-proxy`).
- **Generowanie lokalnego certyfikatu TLS (Ścieżka z certyfikatem):** Utworzono klucze i certyfikaty spełniające wymagane nazewnictwo (`student-*`) oraz zawierające właściwe pola SAN (*Subject Alternative Name*):
 1. Wygenerowano self-signed CA: `student-ca.key` oraz `student-ca.crt`.
 2. Utworzono klucz prywatny serwera i żądanie CSR: `student-server.key` i `student-server.csr`.
 3. Podpisano certyfikat serwera (`student-server.crt`) przy użyciu lokalnego CA oraz rozszerzeń z pliku `server.cnf` (SAN: `DNS:localhost`, `DNS:reverse-proxy.lab`, `IP:127.0.0.1`).
- **Konfiguracja bloku HTTPS (`default.conf`):** Skonfigurowano bloki `server` dla portu 80 (HTTP) oraz portu 443 (HTTPS), podpinając wygenerowane certyfikaty TLS (`student-server.crt`, `student-server.key`) oraz dołączając reguły z `lab_locations.conf`.
- **Weryfikacja skryptem sprawdzającym:** Za pomocą polecenia `./scripts/reload_nginx.sh && ./scripts/check_lab.sh` uruchomiono automatyczny zestaw testów.

3. Wnioski

1. **Wdrożenie Reverse Proxy:** Poprawna konfiguracja przekierowań pozwala na ukrycie infrastruktury backendowej (port 9000) przed bezpośrednim dostępem z zewnątrz, zwiększając bezpieczeństwo aplikacji.
2. **Nagłówki HTTP:** Odpowiednie zarządzanie nagłówkami (`X-Forwarded-Proto`, `X-Real-IP`) jest kluczowe dla zachowania kontekstu klienta, podczas gdy

ukrywanie nagłówków typu **X-Powered-By** utrudnia potencjalnym atakującym identyfikację użytej technologii.

3. **Zabezpieczenie TLS i struktura SAN:** Do poprawnej weryfikacji połączenia szyfrowanego HTTPS przez klientów i skrypty sprawdzające niezbędne jest nie tylko poprawne podpisanie certyfikatu przez CA, ale również uwzględnienie wszystkich domen i adresów IP w rozszerzeniach SAN (*Subject Alternative Name*).